Reviewer Pack — Minimal Rank of Group Representations over DPLL Search Tree …

Entry 4b714d527b92 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 12:12:39 UTC

Minimal Rank of Group Representations over DPLL Search Tree Width

Entry ID: `4b714d527b92` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 12:12:39 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Group Representations) **Field B** (complexity object): Complexity Theory: DPLL Search Tree Width

Statement:

> ['For any group G and its representation V over a field F, the minimal rank of V, denoted as minrank(V), is at least linearly related to the DPLL search tree width of the k-CNF formula corresponding to the boolean function represented by V.', 'Specifically, there exists a constant c > 0 such that for all groups G and their representations V, minrank(V) ≥ c * DPLL_search_tree_width(G).', 'Furthermore, this relationship holds with high probability when considering random k-CNF formulas.']

Rationale (proposer's reasoning):

> ['The connection between group representation theory and computational complexity has been studied in the context of arithmetic circuits. This conjecture aims to extend this connection to the realm of DPLL search tree width.', 'Group representations provide a natural way to encode boolean functions, which are fundamental to understanding the complexity of SAT and other NP problems. The minimal rank could capture essential structural properties of these representations that are relevant to their computational complexity.', 'Exploring this relationship might reveal new insights into the nature of complexity classes or provide new tools for analyzing DPLL search trees.']

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `46e9a4744f1b60e3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given group representation V and its corresponding k-CNF formula, we consider the conjecture supported if the ratio of minrank(V) to DPLL_search_tree_width(G) is greater than or equal to c (where c > 0 is a constant), AND this relationship holds for at least 80%

of seeds tested. The criterion is falsified if any seed produces a ratio less than c, or if any seed fails to meet the support threshold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random group G and its representation V over a field F
    n = random.randint(5, 40)
    G = list(range(n))
    V = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]

    # Calculate the minimal rank of V
    minrank_V = sum(1 for row in V if any(row))  # Count non-zero rows

    # Generate a random k-CNF formula representing the boolean function represented by V
    k = random.randint(2, n-1)
    num_clauses = random.randint(n, 2*n)
    clauses = []
    for _ in range(num_clauses):
        clause = [random.choice(G) for _ in range(k)]
        clauses.append(clause)

    # Calculate the DPLL search tree width of the k-CNF formula
    def dpll_width(clauses):
        if not clauses:
            return 0
        if any(len(clause) == 1 for clause in clauses):
            return max(dpll_width([c for c in clauses if len(c) > 1]) for c in set(sum(clauses, []
        unit_clause = next((c for c in clauses if len(c) == 1), None)
```

```
        if unit_clause:
            return 1 + dpll_width([c for c in clauses if unit_clause[0] not in c])
        else:
            p = random.choice(G)
            return max(dpll_width([c for c in clauses if p not in c]), dpll_width([c for c in clau

    DPLL_search_tree_width_G = dpll_width(clauses)

    # Correlate the two measures
    ratio = minrank_V / (DPLL_search_tree_width_G + 1e-9)  # Avoid division by zero

    return {
        "metric_name": "minrank_to_DPLL_width_ratio",
        "metric_value": ratio,
        "instances_tested": n,
        "conjecture_holds": ratio >= 0.5,  # Placeholder for actual constant c
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(30)]  # Default to first 30

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    std_ratio = math.sqrt(sum((result["metric_value"] - mean_ratio)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='minrank_to_DPLL_width_ratio' first_failing_seed=
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb26548.py", line 68, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb26548.py", line 50, in run_trial
    DPLL_search_tree_width_G = dpll_width(clauses)
                               ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb26548.py", line 48, in dpll_width
```

```
   return max(dpll_width([c for c in clauses if p not in c]), dpll_width([c for c in clauses if -
p not in c]))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb26548.py", line 48, in dpll_width
   return max(dpll_width([c for c in clauses if p not in c]), dpll_width([c for c in clauses if -
p not in c]))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb26548.py", line 48, in dpll_width
   return max(dpll_width([c for c in clauses if p not in c]), dpll_width([c for c in clauses if -
p not in c]))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 993 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb26548.py", line 47, in dpll_width
    p = random.choice(G)
        ^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 348, in choice
    return seq[self._randbelow(len(seq))]
               ^^^^^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide evidence for the conjecture. | next: Re-examine the code for potential recursion issues and ensure that the test can run to completion. If the issue is resolved, re-run the test with a larger number of seeds to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13043
2	preregistration	ollama_remote	glm4:latest	0	0	6073
3	novelty	ollama_remote	glm4:latest	0	0	4739
4	novelty	ollama_remote	glm4:latest	0	0	5118
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15920
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12085
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10180
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9343
9	judge	ollama_remote	glm4:latest	0	0	13255

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 89756 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4b714d527b92.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4b714d527b92.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4b714d527b92.tar.gz` (if generated)